

CondorNet v4.3 & Datasets v4.3 Model Training Guide and Args Definitions

Below is a **one-by-one breakdown of the model-specific arguments** in the exact command I am running right now, with:

- what the arg controls in the model,
- how it is used computationally,
- the mathematics it parameterizes,
- and why it exists in CondorNet v4.3 (in your current gate-stability context).

I'm treating "model-specific" as: **architecture dimensions, predicate/set/superset topology, gate temperature, convergence subsystem, and gate regularizers**. (I will not re-document generic trainer args like `--lr`, `--epochs`, `--batch-size`, etc.)

The Command:

```
⚡ main ~/spy-iron-condor-trading python intelligence/condor_train_net_v43.py \

--data-dir data/Datasetv4/v43/ --m1-file m1_dataset_v43_final.csv --m5-file m5_dataset_v43_final.csv

--m15-file m15_dataset_v43_final.csv --h1-file h1_dataset_v43_final.csv --options-file options_2025_v43.csv \

--d-joint 128 --epochs 22 --batch-size 256 --lr 1e-4 --lookback 200 --accum-steps 2 --patience 20 --verbose \

--d-h 128 --d-v 16 --d-m 32 --d-r 16 --d-control 64 --n-predicates 64 --n-sets 32 --n-super-sets 16 \

--convergence-threshold 0.05 --convergence-patience 20 --gate-temp 3 --diversity-alpha 0.005 --diversity-std-target 0.14 \

--logit-var-alpha 0.10 --logit-var-target 2.0 --logit-iqr-alpha 0.08 --logit-iqr-target 1.0 \

--logit-eps 1e-4 --logit-mad-alpha 0.05 --logit-mad-target 0.9 --logit-warmup-frac 0.0 \

--output models/condor_net_v43.pth \

> train_v43_t4_tenth_test.log 2>&1 &
```

[1] 265202

```
⚡ main ~/spy-iron-condor-trading ptail -f train_v43_t4_tenth_test.log
```

CONDORNET v4.3 TRAINING LABORATORY

Version: CNv43_DSv43 | Schema: v4.3.0

=====

...

A. Latent Geometry / Architectural Dimensions

1) --d-h 128 (Hidden / state latent dimension)

What it is

The dimensionality of the primary latent state vector used by the core dynamical model (your continuous-time / ETD-like state evolution).

How it is used

The model maintains a latent state:

$$h_t \in \mathbb{R}^{d_h}$$

and updates it via a generator-driven dynamics (conceptually):

$$h_{t+\Delta t} = \exp(A_t \Delta t) h_t + \Delta t \phi_1(A_t \Delta t) B_t u_t + (\text{forcing terms})$$

Definitions

- h_t — hidden latent state at time t
- d_h — hidden dimension (d-h)
- A_t — state generator matrix (possibly conditioned on controls)
- Δt — time step
- $\phi_1(X)$ — ETD-1 phi operator (defined later)
- B_t — control injection operator
- u_t — control input / context vector

Why it exists

d_h controls **capacity of continuous-time memory**. Too small and the state cannot encode multi-regime dynamics; too large and you over-parameterize, amplify instability, and increase compute (notably Aoperations). Your current setting (128) is a “mid-size” that has empirically supported stable training on T4.

2) --d-v 16 (Visible / predicate-visible embedding dimension)

What it is

The embedding width for the “visible” predicate-facing channel—i.e., the representation feeding predicate gates before sets/supersets.

How it is used

Predicates typically take a feature-derived vector and map to predicate activations:

$$p_t = \sigma(W_p v_t + b_p)$$

with

$$v_t \in \mathbb{R}^{d_v}, p_t \in (0,1)^{n_{\text{pred}}}$$

Definitions

- v_t — visible embedding at time t
- d_v — visible dimension (d-v)
- W_p — predicate projection
- b_p — bias
- p_t — predicate activations
- n_{pred} — number of predicates

Why it exists

You want an information bottleneck before predicate logic so the predicate layer learns **structured, compressible signals** (pivot logic, inequality structure, etc.) rather than brute-force memorization.

3) --d-m 32 (Memory / mixture / intermediate dimension)

What it is

A middle-layer width used for mixing, gating, or intermediate transforms between the latent state and predicate logic (often a “model” width distinct from d_h).

How it is used

Commonly:

$$m_t = f(W_m h_t + b_m), m_t \in \mathbb{R}^{d_m}$$

and then downstream logic consumes m_t (or concatenations).

Definitions

- m_t — intermediate representation
- d_m — intermediate dimension (d-m)
- $f(\cdot)$ — nonlinearity (e.g., GELU / tanh)

Why it exists

This is a **control knob for feature abstraction** between dynamics and logic. It prevents the predicate system from being forced to read raw h_t directly (which tends to be entangled and harder to interpret).

4) --d-r 16 (Relational dimension)

What it is

The width of the relational logic representation used inside your hierarchical gate and relation operators.

How it is used

A relational module takes supersets S and produces a relation score:

$$r = R(S) \in \mathbb{R}^{d_r}$$

Then a scalar (or low-dim) logit is formed, e.g.:

$$\tilde{z} = w^T r + b$$

Definitions

- S — supersets tensor
- $R(\cdot)$ — relational map
- r — relational embedding
- d_r — relational dimension (d-r)
- w, b — projection weights / bias
- $\tilde{z}$ — pre-centered gate logit before temperature

Why it exists

This is your “logic bottleneck” for gating. Keeping d_r small makes the gate depend on **low-dimensional, interpretable relational structure** rather than high-dimensional noise.

5) --d-control 64 (Control input dimension)

What it is

The width of the control vector u_t injected into dynamics and/or gating.

How it is used

You can model conditioning as:

$$A_t = A_\theta(u_t), B_t = B_\theta(u_t)$$

or at minimum:

$$u_t \in \mathbb{R}^{d_{\text{control}}}$$

and used as input to modulators or adapters.

Definitions

- u_t — control vector at time t
- d_{control} — control dimension (d-control)
- $A_{\theta}(\cdot)$ — conditioned generator function
- $B_{\theta}(\cdot)$ — conditioned injection function

Why it exists

Options trading is regime-conditioned (IVR, trend, volatility energy, pivot strength). A control vector allows the generator and logic to adapt without exploding parameter count.

B. Predicate / Set / Superset Topology

6) --n-predicates 64

What it is

Number of learned predicate units.

How it is used

At each time (or per sequence), the model computes predicates:

$$p \in (0,1)^{n_{\text{pred}}}$$

Typically with a sigmoid gate:

$$p_j = \sigma(a_j), a = W_p v + b_p$$

Definitions

- n_{pred} — number of predicates
- p_j — predicate activation for predicate j
- a_j — predicate logit

Why it exists

Predicates are the atomic logical units (inequalities, pivot conditions, regime flags). Increasing n-predicates increases logic vocabulary but risks redundancy/collapse. 64 is a reasonable fixed dictionary size.

7) --n-sets 32

What it is

Number of sets built from predicates.

How it is used

Sets are soft logical groupings:

$$s_i = g_i(p), s_i \in (0,1)^{d_{\text{set}}}$$

In many neural logic systems, a set activation is a learned soft-AND/OR composition. A common differentiable AND is:

$$\text{AND}(p_{j_1}, \dots, p_{j_k}) \approx \prod_{\ell=1}^k p_{j_\ell}$$

or log-space:

$$\log \text{AND} \approx \sum_{\ell=1}^k \log(p_{j_\ell})$$

Definitions

- s_i — set i representation
- p — predicate vector
- $g_i(\cdot)$ — set constructor
- n_{set} — number of sets

Why it exists

Sets correspond to mid-level trading logic chunks (e.g., “trend + breakout + IVR high”). They’re the bridge between raw predicates and supersets used by routing.

8) --n-super-sets 16

What it is

Number of supersets built from sets; these feed hierarchical gate logic.

How it is used

Supersets concatenate or aggregate set outputs:

$$S = \text{concat}(S_1, \dots, S_{n_{\text{sup}}}) \in \mathbb{R}^{n_{\text{sup}} \times d_r}$$

Then gating uses relational logic over S .

Definitions

- n_{sup} — number of supersets
- S_i — supersets
- S — stacked / concatenated supersets

Why it exists

Supersets are the top-level “logic capsules” used for routing decisions (entry/exit, strategy family, sizing regime). 16 provides enough mixture diversity without making relational comparison expensive.

C. Convergence Subsystem (Trainer-Integrated but Model-Critical)

9) --convergence-threshold 0.05

What it is

A convergence detector threshold that triggers early stopping within an epoch and/or snapshot restore behavior (your “CONVERGENCE stopped at batch ... restoring snapshot” behavior).

How it is used

Define a convergence statistic, typically based on loss improvement:

$$\Delta_k = \frac{|\ell_k - \ell_{k-1}|}{\max(\ell_{k-1}, \delta)}$$

Trigger if:

$$\Delta_k < \tau_c$$

Definitions

- ℓ_k — loss at batch k
- Δ_k — normalized improvement measure
- δ — small constant to avoid division by zero
- τ_c — convergence threshold (convergence-threshold)

Why it exists

It saves compute and reduces over-optimization within an epoch once the model enters a stable region. However, your Run 9 evidence suggests it can bias toward “best-batch minima” and hurt generalization—hence your next controlled ablation (remove convergence-active).

10) --convergence-patience 20

What it is

Number of consecutive batches meeting the convergence condition before triggering the convergence action.

How it is used

Convergence triggers if:

$$\sum_{i=k-P+1}^k \mathbf{1}[\Delta_i < \tau_c] = P$$

Definitions

- P — patience (convergence-patience)
- $\mathbf{1}[\cdot]$ — indicator function

Why it exists

Prevents false triggers due to stochastic batch noise.

11) --convergence-active

What it is

Switch that enables the convergence action (stop early, restore snapshot).

How it is used

If enabled, the trainer interrupts epoch progression and restores the best-known in-epoch parameters:

$$\theta \leftarrow \theta_{\text{epoch}}^*$$

Definitions

- θ — model parameters
- θ_{epoch}^* — best snapshot within the epoch

Why it exists

It's an aggressive generalization heuristic—useful when late-epoch drift is overfit, but can also lock the model into a narrow basin.

D. Gate Temperature

12) --gate-temp 3

What it is

Initial value of the gate temperature parameter τ .

How it is used

The gate logit is scaled:

$$\lambda = \sigma(\tau(z - \mathbb{E}[z]))$$

Definitions

- λ — gate probability
- σ — sigmoid
- τ — temperature (gate-temp)
- z — raw relational gate score
- $\mathbb{E}[z]$ — batch mean centering

Why it exists

τ controls the sharpness of routing. Higher τ yields harder routing (more 0/1), lower τ yields softer routing. You empirically observed τ often stagnates; Run 9 shows slight movement once the gate is stabilized.

E. Gate Regularization (Run 9 Stabilizers)

These were added because the architecture repeatedly collapses into degenerate routing attractors unless constrained.

13) --diversity-alpha 0.005

Coefficient for lambda-std hinge:

$$\mathcal{L}_\lambda = \alpha_\lambda \cdot \text{softplus} \left(\frac{\sigma_{\lambda, \text{target}} - \text{std}(\lambda)}{\epsilon} \right) \epsilon$$

Definitions

- α_λ —diversity-alpha
- $\sigma_{\lambda,\text{target}}$ —diversity-std-target
- ϵ —smoothing scale

Why it exists

Demoted to a “sanity rail” because std (λ) can be misled by outliers; the real control is in z-space now.

14) --diversity-std-target 0.14

The minimum target dispersion for λ :

$$\text{std}(\lambda) \geq 0.14$$

Definitions

- $\text{std}(\lambda)$ —batch standard deviation of routing probabilities

Why it exists

Prevents λ from collapsing to near-constant (e.g., always 0.5 or always 0/1).

15) --logit-var-alpha 0.10

Coefficient for z-variance floor:

$$\mathcal{L}_{z,\text{var}} = \alpha_z \cdot \text{softplus} \left(\frac{v_{\text{target}} - \text{Var}(z)}{\epsilon} \right) \epsilon$$

Definitions

- α_z —logit-var-alpha
- v_{target} —logit-var-target

Why it exists

Directly prevents global contraction of z, which you identified as the true collapse mechanism upstream of λ .

16) --logit-var-target 2.0

Minimum target variance:

$$\text{Var}(z) \geq 2.0$$

Why it exists

Calibrated to keep zdispersion in the regime where healthy gates existed (z_std around 1.4–2.0).

17) --logit-iqr-alpha 0.08

Coefficient for IQR floor:

$$\mathcal{L}_{z,\text{IQR}} = \alpha_{\text{IQR}} \cdot \text{softplus}\left(\frac{\Delta_{\text{target}} - \text{IQR}(z)}{\epsilon}\right) \epsilon$$

Definitions

- α_{IQR} —logit-iqr-alpha
- $\text{IQR}(z) = Q_{75}(z) - Q_{25}(z)$

Why it exists

Variance can be satisfied by extreme outliers; IQR forces the **bulk** to spread—this targets your “degenerate bimodal spike” pathology.

18) --logit-iqr-target 1.0

Target bulk spread:

$$Q_{75}(z) - Q_{25}(z) \geq 1.0$$

Why it exists

Prevents “50% of logits within ± 0.012 ” degeneracy you observed.

19) --logit-eps 1e-4

Numerical clamp parameter for logit recovery:

$$\begin{aligned} p &= \text{clamp}(\lambda, \epsilon, 1 - \epsilon) \\ z &= \log(p) - \log(1 - p) \end{aligned}$$

Definitions

- ϵ — logit-eps
- p — clamped probability

Why it exists

Avoids $\pm\infty$ logits when λ saturates near 0 or 1; also makes gradients finite.

20) --logit-mad-alpha 0.05

Coefficient for MAD floor:

$$\mathcal{L}_{z,\text{MAD}} = \alpha_{\text{MAD}} \cdot \text{softplus} \left(\frac{m_{\text{target}} - \mathbb{E}[|z|]}{\epsilon} \right) \epsilon$$

Why it exists

Even with healthy variance/IQR, the distribution can shrink toward 0 symmetrically in some regimes; MAD prevents “collapse-to-zero” mass.

21) --logit-mad-target 0.9

Target absolute magnitude:

$$\mathbb{E}[|z|] \geq 0.9$$

Why it exists

Encourages meaningful gate logits (not hovering near 0).

22) --logit-warmup-frac 0.0

Warmup fraction for z-regularizers.

Interpretation:

$$\alpha(t) = \begin{cases} \alpha_{\text{max}}, & \text{if warmup} = 0 \\ \alpha_{\text{max}} \cdot \min(1, t/t_{\text{warm}}), & \text{otherwise} \end{cases}$$

Definitions

- $\alpha(t)$ — time-varying regularizer weight
- t — training progress index (epoch or step)

- t_{warm} — warmup horizon

Why it exists

Empirically, collapse was happening within epoch 1; warmup allowed “free collapse.” Setting 0.0 forces full strength immediately.

F. Output Artifact

23) `--output models/condor_net_v43.pth`

Path where the best checkpoint is stored. Model selection uses the lowest validation loss.